

Dynamic Programming DSA Handbook

A Complete, Practical Guide to DP Patterns in Python

PART 1: DP FUNDAMENTALS

1.1 What is Dynamic Programming?

1.2 Core Terminology

1.3 Top-Down vs Bottom-Up

1.4 How to Design a DP Solution

1.5 How to Read This Handbook

PART 2: PATTERNS

2.1 1D / Linear DP

2.2 2D Grid DP

2.3 0/1 Knapsack

2.4 Unbounded Knapsack & Coin Change

2.5 Longest Common Subsequence (LCS) Family

2.6 Longest Increasing Subsequence (LIS) Family

2.7 Palindrome & String DP

2.8 Interval DP

2.9 DP on Trees

2.10 Bitmask DP

PART 3: QUICK REFERENCE CHEAT SHEET

Decision Guide — “Which DP Pattern Do I Use?”

PART 1: FUNDAMENTALS

1.1 What is Dynamic Programming?

Dynamic Programming (DP) solves a problem by breaking it into overlapping subproblems, solving each subproblem only once, and reusing that result wherever it's needed again. It is recursion (or careful iteration) PLUS a cache. DP applies exactly when two properties hold: **optimal substructure** (the optimal answer to the full problem can be built from optimal answers to its subproblems) and **overlapping subproblems** (the same subproblem is encountered again and again via different paths).

1.2 Core Terminology

State	The set of parameters that uniquely identify a subproblem (e.g., index i , remaining capacity, previous index chosen).
Transition / Recurrence	The formula expressing a state's answer in terms of smaller/previous states (e.g., $dp[i] = dp[i-1] + dp[i-2]$).
Base Case	The smallest state(s) whose answer is known directly, without needing a transition.
Optimal Substructure	The property that an optimal solution to the whole problem is composed of optimal solutions to its subproblems.
Overlapping Subproblems	The property that naive recursion revisits the exact same state many times — the reason caching helps at all.
Memoization	Top-down DP: write the natural recursion, cache each state's result the first time it's computed.
Tabulation	Bottom-up DP: iteratively fill a table (array) of states in an order that guarantees dependencies are computed first.
State Space	The full set of distinct states the DP could visit — its size directly determines the DP's time/space complexity.
Space Optimization	Reducing a DP table's dimensions (e.g., $2D \rightarrow 1D$) when each state only depends on the immediately previous row/layer.

1.3 Top-Down vs Bottom-Up

Top-Down (Memoized Recursion)	Write the recursive solution as if you had infinite time, add a cache dict/array, check-before-compute. Naturally only visits states that are actually needed.
Bottom-Up (Tabulation)	Identify a valid fill order (usually smallest state first), iterate through all states filling a table. Avoids recursion-depth limits and function-call overhead; usually faster in Python.
When to prefer top-down	When the state space is large but only a fraction of states are actually reachable/needed, or when the recursive definition is much clearer than the iterative fill order.

When to prefer bottom-up	When you need every state anyway, want to space-optimize (rolling array), or need to avoid Python's recursion limit for large inputs.
---------------------------------	---

1.4 How to Design a DP Solution

- **1. Define the state** — what does `dp[...]` represent, in one precise sentence? (e.g., "`dp[i]` = length of the longest increasing subsequence ending exactly at index `i`")
- **2. Find the recurrence** — how does `dp[state]` relate to `dp[smaller states]`? Try small examples by hand.
- **3. Identify base cases** — the states you can answer with no further breakdown.
- **4. Decide the fill order** — for bottom-up, make sure every dependency of `dp[state]` is computed before `dp[state]` itself.
- **5. Optimize space** (optional) — if `dp[i]` only depends on `dp[i-1]` (or a small window), collapse the table to $O(1)$ or $O(k)$ variables.

1.5 How to Read This Handbook

For every pattern below you will get, in this exact order: (1) What it's for, (2) Use Cases, (3) Limitations, (4) Time & Space Complexity, (5) Approach (short) — state definition + recurrence, (6) Keywords & Constraints that hint you should use this pattern, (7) Fully commented Python code, and (8) Practice Questions with a short reasoning of why the pattern fits.

PART 2: PATTERNS & ALGORITHMS

2.1 1D / Linear DP

What it's for: DP where the state is a single index into a sequence, and each state's answer depends on a small, fixed set of earlier indices. The simplest and most common DP shape — the on-ramp to every other pattern.

Use Cases:

- Fibonacci-style counting (climbing stairs, tiling problems).
- House Robber — maximize value picking non-adjacent elements.
- Maximum subarray sum (Kadane's algorithm), best time to buy/sell stock.
- Any "answer at position i depends on answer(s) at $i-1$ and/or $i-2$ " problem.

Limitations:

- Only fits problems whose state truly reduces to "position in a single sequence" — if the answer depends on more context (which items chosen, remaining capacity), you need a richer state (see Knapsack, LIS).
- A naive recursive version without memoization degenerates into exponential time for branching recurrences like Fibonacci.
- It's easy to conflate "value at index i " with "best answer using the first i elements" — be precise about which one $dp[i]$ means.

Time Complexity: $O(n)$ — one recurrence evaluation per index **Space Complexity:** $O(n)$ for the table, reducible to $O(1)$ or $O(k)$ if only the last few states are needed

Approach (short): Define $dp[i]$ as the answer considering the first i elements (or ending exactly at index i). Express $dp[i]$ using $dp[i-1]$ (and sometimes $dp[i-2]$ or further back), fill left to right, base case $dp[0]/dp[1]$.

Keywords & Constraints: "climbing stairs", "maximum subarray", "house robber", "in terms of previous 1-2 states", $n \leq 10^5$ - 10^6 .

```
def climb_stairs(n):
    """Ways to reach step n taking 1 or 2 steps at a time. dp[i] = ways to reach step i."""
    if n <= 2:
        return n
    prev2, prev1 = 1, 2 # dp[1], dp[2]
    for i in range(3, n + 1):
        prev2, prev1 = prev1, prev1 + prev2 # dp[i] = dp[i-1] + dp[i-2], rolled into 2 variables
    return prev1

def house_robber(nums):
    """dp[i] = max money robbing houses[0..i] without robbing two adjacent houses."""
    rob, skip = 0, 0 # rob = best including house i-1, skip = best excluding it
    for val in nums:
        rob, skip = skip + val, max(rob, skip) # new_rob takes val + old best-excluding-prev
    return max(rob, skip)

def max_subarray(nums):
    """Kadane's algorithm: dp[i] = best subarray sum ENDING exactly at i."""
    best = current = nums[0]
    for x in nums[1:]:
        current = max(x, current + x) # either extend previous subarray, or start fresh at x
        best = max(best, current)
    return best

def best_time_to_buy_sell_stock(prices):
    """Single transaction: track min price so far, max profit so far."""
    min_price, max_profit = float('inf'), 0
    for p in prices:
```

```

    min_price = min(min_price, p)
    max_profit = max(max_profit, p - min_price)
return max_profit

```

Practice Questions — 1D / Linear DP

Q1. Climbing Stairs Count ways to reach step n taking 1 or 2 steps at a time. *Approach:* $dp[i] = dp[i-1] + dp[i-2]$; base cases $dp[1]=1$, $dp[2]=2$ — Fibonacci in disguise.

Q2. House Robber Maximize money robbed from houses without robbing two adjacent ones. *Approach:* $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$ — either skip house i or take it plus the best excluding its neighbor.

Q3. Maximum Subarray (Kadane's Algorithm) Find the maximum sum of a contiguous subarray. *Approach:* $dp[i] = \max(\text{nums}[i], dp[i-1] + \text{nums}[i])$ — at each index decide whether to extend the running subarray or restart.

Q4. Best Time to Buy and Sell Stock Maximize profit from one buy and one sell. *Approach:* Track the minimum price seen so far and the best profit achievable selling at the current price — a rolling single-variable DP.

Q5. Decode Ways Count ways to decode a digit string into letters ($A=1\dots Z=26$). *Approach:* $dp[i] = dp[i-1]$ (if single digit valid) + $dp[i-2]$ (if two-digit valid) — same shape as climbing stairs with validity checks added.

Q6. Delete and Earn Maximize points by deleting a number (earning its value) which also removes adjacent-value numbers. *Approach:* Convert to a House-Robber-shaped problem over value buckets, since taking value v forbids taking $v-1$ and $v+1$ — sum counts per value, then apply the rob/skip recurrence.

2.2 2D Grid DP

What it's for: DP over a 2D grid where $dp[r][c]$ depends on neighboring cells (typically the cell above and the cell to the left), used for path counting, path cost minimization, and grid-based optimization.

Use Cases:

- Counting the number of distinct paths through a grid with movement restrictions.
- Minimum/maximum cost path from one corner of a grid to another.
- Grid problems with obstacles, weighted cells, or triangle-style layouts.
- Any 2D matrix problem naturally described as "coming from above or from the left".

Limitations:

- Space is $O(\text{rows} \times \text{cols})$ unless optimized — since each row typically only depends on the previous row, this is a classic candidate for rolling-array space reduction to $O(\text{cols})$.
- Obstacles/blocked cells need explicit handling (set that dp value to 0 or infinity depending on count vs. min-cost) or the recurrence silently produces wrong answers.
- Diagonal-movement variants change the recurrence (3-way instead of 2-way max/sum) — always re-derive from the problem's actual allowed moves.

Time Complexity: $O(\text{rows} \times \text{cols})$ — one recurrence evaluation per cell **Space Complexity:** $O(\text{rows} \times \text{cols})$, reducible to $O(\text{cols})$ with a rolling 1D array

Approach (short): Define $dp[r][c]$ as the answer (count of paths, or min/max cost) to reach cell (r, c) . Fill row by row (or column by column), computing each cell from the cell(s) that can reach it (above, left, sometimes diagonal), with the first row/column as base cases.

Keywords & Constraints: "unique paths", "minimum path sum", "grid", "matrix", $\text{rows} \times \text{cols} \leq 10^6$ typically.

```
def unique_paths(rows, cols):
    """Count paths from top-left to bottom-right moving only right or down."""
    dp = [1] * cols                    # first row: only 1 way to reach any cell (all rights)
    for r in range(1, rows):
        for c in range(1, cols):
            dp[c] += dp[c - 1]         # dp[c] (from above, still old value) + dp[c-1] (from left, new value)
    return dp[-1]

def min_path_sum(grid):
    """Minimum cost path from top-left to bottom-right, moving right or down."""
    rows, cols = len(grid), len(grid[0])
    dp = [[0] * cols for _ in range(rows)]
    dp[0][0] = grid[0][0]
    for c in range(1, cols):
        dp[0][c] = dp[0][c - 1] + grid[0][c]          # first row: only reachable from the left
    for r in range(1, rows):
        dp[r][0] = dp[r - 1][0] + grid[r][0]          # first column: only reachable from above
    for r in range(1, rows):
        for c in range(1, cols):
            dp[r][c] = grid[r][c] + min(dp[r - 1][c], dp[r][c - 1])
    return dp[-1][-1]

def unique_paths_with_obstacles(grid):
    """Same as unique_paths, but obstacle cells (1) contribute 0 paths."""
    rows, cols = len(grid), len(grid[0])
    dp = [[0] * cols for _ in range(rows)]
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:
                dp[r][c] = 0                    # obstacle: unreachable
            elif r == 0 and c == 0:
                dp[r][c] = 1
            else:
```

```

        above = dp[r - 1][c] if r > 0 else 0
        left = dp[r][c - 1] if c > 0 else 0
        dp[r][c] = above + left
    return dp[-1][-1]

```

Practice Questions — 2D Grid DP

Q1. Unique Paths Count paths from top-left to bottom-right moving right or down. *Approach:* $dp[r][c] = dp[r-1][c] + dp[r][c-1]$; first row and column are all 1 since there's exactly one way to reach them.

Q2. Unique Paths II (With Obstacles) Same as Unique Paths but some cells are blocked. *Approach:* Same recurrence, but any obstacle cell is forced to $dp = 0$, correctly cutting off paths through it.

Q3. Minimum Path Sum Find the minimum-cost path through a weighted grid. *Approach:* $dp[r][c] = grid[r][c] + \min(dp[r-1][c], dp[r][c-1])$ — same shape as Unique Paths with min+add instead of sum.

Q4. Maximal Square Find the largest square of 1s in a binary matrix. *Approach:* $dp[r][c] = \text{size of the largest square with bottom-right corner at } (r,c) = 1 + \min(dp[r-1][c], dp[r][c-1], dp[r-1][c-1])$ if $grid[r][c] == 1$, else 0.

Q5. Triangle Minimum Path Sum Find the minimum path sum from top to bottom of a triangle array. *Approach:* $dp[c]$ at row $r = triangle[r][c] + \min(dp[c], dp[c+1])$ computed bottom-up, reusing a single rolling row array.

Q6. Dungeon Game Find the minimum initial health needed to rescue the princess through a grid of gains/losses. *Approach:* Work BACKWARDS from the bottom-right cell: $dp[r][c] = \max(1, \min(dp[r+1][c], dp[r][c+1]) - grid[r][c])$ — shows grid DP can fill in reverse when the natural dependency runs backward.

2.3 0/1 Knapsack

What it's for: Choosing a subset of items, each usable AT MOST ONCE, to maximize value under a capacity/weight constraint. The state tracks both "how many items considered" and "capacity used so far".

Use Cases:

- Classic knapsack: maximize value of items packed within a weight limit.
- Subset Sum — can a subset of numbers sum to exactly a target?
- Equal Subset Sum Partition — can the array be split into two equal-sum halves?
- Target Sum — assign +/- signs to reach a target (reframed as a subset-sum variant).

Limitations:

- Full 2D table is $O(n \times \text{capacity})$ time and space — can be too slow/memory-heavy if capacity is very large (e.g., capacity up to 10^9 needs a different approach entirely, like meet-in-the-middle).
- The "each item used at most once" constraint requires iterating capacity in DECREASING order when space-optimizing to 1D, or you'll accidentally reuse an item within the same row (that's Unbounded Knapsack instead).
- Doesn't directly return WHICH items were chosen unless you additionally track choices or backtrack through the filled table.

Time Complexity: $O(n \times \text{capacity})$ — n items, each considered once per capacity value **Space Complexity:** $O(n \times \text{capacity})$, reducible to $O(\text{capacity})$ with a 1D rolling array (iterated capacity high-to-low)

Approach (short): Define $dp[i][w]$ = best value using the first i items with capacity w . For each item, either skip it ($dp[i-1][w]$) or take it ($value[i] + dp[i-1][w - weight[i]]$, if it fits) and take the better option.

Keywords & Constraints: "each item used at most once", "subset that sums to target", "partition into two equal subsets", n and capacity both $\leq \sim 10^3$ - 10^4 for the $O(n \times \text{capacity})$ table to be feasible.

```
def knapsack_01(weights, values, capacity):
    """Maximum value picking each item at most once, within capacity."""
    n = len(weights)
    dp = [0] * (capacity + 1)          # 1D rolling array over capacity
    for i in range(n):
        for w in range(capacity, weights[i] - 1, -1): # iterate HIGH to LOW: prevents reusing item i twice
            dp[w] = max(dp[w], values[i] + dp[w - weights[i]])
    return dp[capacity]

def subset_sum_exists(nums, target):
    """Can any subset of nums sum to exactly target?"""
    dp = [False] * (target + 1)
    dp[0] = True                      # empty subset sums to 0
    for num in nums:
        for w in range(target, num - 1, -1): # high to low, same 0/1 reasoning as knapsack
            dp[w] = dp[w] or dp[w - num]
    return dp[target]

def can_partition_equal_subset_sum(nums):
    """Can nums be split into two subsets with equal sum?"""
    total = sum(nums)
    if total % 2 != 0:
        return False
    return subset_sum_exists(nums, total // 2)

def target_sum_ways(nums, target):
    """Count ways to assign +/- to each num so the total equals target.
    Reframed: let P = sum of '+' subset, N = sum of '-' subset. P - N = target, P + N = total.
    So P = (total + target) / 2 -> count subsets summing to P."""
    total = sum(nums)
```

```

if (total + target) % 2 != 0 or total < abs(target):
    return 0
P = (total + target) // 2
dp = [0] * (P + 1)
dp[0] = 1
for num in nums:
    for w in range(P, num - 1, -1):
        dp[w] += dp[w - num]
return dp[P]

```

Practice Questions — 0/1 Knapsack

Q1. 0/1 Knapsack Maximize value of items packed within a weight limit, each item used at most once. *Approach:* $dp[w] = \max(dp[w], \text{value}[i] + dp[w - \text{weight}[i]])$ iterated capacity high-to-low per item to enforce single use.

Q2. Subset Sum Determine if any subset of an array sums to exactly a target. *Approach:* Boolean version of 0/1 knapsack: $dp[w] = dp[w] \text{ OR } dp[w - \text{num}]$, same high-to-low iteration to avoid reusing an element.

Q3. Partition Equal Subset Sum Determine if an array can be split into two subsets of equal sum. *Approach:* Reduces directly to Subset Sum with target = total/2 (and immediately impossible if total is odd).

Q4. Target Sum Count ways to assign + or - to each number to reach a target sum. *Approach:* Algebraic reframing turns +/- assignment into a subset-sum counting problem: find subsets summing to $(\text{total} + \text{target})/2$.

Q5. Last Stone Weight II Minimize the final stone weight after repeatedly smashing pairs together. *Approach:* Equivalent to partitioning stones into two groups as evenly as possible — find the subset sum closest to total/2, answer is total - 2×that sum.

Q6. Ones and Zeroes Find the max number of strings selectable given limited total 0s and m and 1s and n. *Approach:* 2D-capacity 0/1 knapsack: $dp[\text{zeros}][\text{ones}] = \text{max count}$, iterating both capacity dimensions high-to-low per string to enforce single use.

2.4 Unbounded Knapsack & Coin Change

What it's for: Like 0/1 Knapsack, but each item/denomination can be used UNLIMITED times. The key structural difference: capacity is iterated low-to-high (not high-to-low), allowing the same item to be reused within the same pass.

Use Cases:

- Coin Change — minimum coins, or number of ways, to make a target amount.
- Rod Cutting — maximize revenue cutting a rod into pieces sold at given prices.
- Unbounded Knapsack — maximize value when items can be picked any number of times.
- Any "combinations/ways using unlimited supply of each option" counting problem.

Limitations:

- Iterating capacity low-to-high (to allow reuse) versus high-to-low (0/1, no reuse) is the single most common DP bug in this family — always double-check which one the problem requires.
- Counting COMBINATIONS (order doesn't matter) vs. PERMUTATIONS (order matters, e.g., "ways to climb stairs with steps of size in coins") requires swapping the loop nesting order (coins outer vs. amount outer) — easy to get backwards.
- Same $O(n \times \text{capacity})$ time/space characteristics and limits as 0/1 Knapsack.

Time Complexity: $O(n \times \text{amount})$ — n denominations/items, each capacity value considered per item **Space**

Complexity: $O(\text{amount})$ with a 1D rolling array

Approach (short): Define $\text{dp}[\text{amount}]$ = best value/count/min-count using unlimited copies of the given items. Iterate amount from low to high (allowing the same item to be picked again within the same fill), unlike 0/1 Knapsack's high-to-low iteration.

Keywords & Constraints: "unlimited supply", "each coin/item may be used any number of times", "minimum coins", "number of ways to make change", $\text{amount} \leq 10^4\text{-}10^5$.

```
def coin_change_min(coins, amount):
    """Minimum number of coins to make amount; -1 if impossible."""
    dp = [0] + [float('inf')] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a - c] + 1)    # low-to-high fill lets the SAME coin be reused
    return dp[amount] if dp[amount] != float('inf') else -1

def coin_change_ways(coins, amount):
    """Number of distinct COMBINATIONS (order doesn't matter) of coins summing to amount."""
    dp = [0] * (amount + 1)
    dp[0] = 1
    for c in coins:
        for a in range(c, amount + 1):
            dp[a] += dp[a - c]
    return dp[amount]

def combination_sum_iv_permutations(nums, target):
    """Number of PERMUTATIONS (order matters) of nums summing to target."""
    dp = [0] * (target + 1)
    dp[0] = 1
    for a in range(1, target + 1):
        for num in nums:
            if num <= a:
                dp[a] += dp[a - num]
    return dp[target]

def rod_cutting_max_revenue(prices, n):
```

```

"""Maximum revenue cutting a rod of length n given price[i] for a piece of length i (1-indexed prices list)."""
dp = [0] * (n + 1)
for length in range(1, n + 1):
    for cut in range(1, length + 1):
        dp[length] = max(dp[length], prices[cut - 1] + dp[length - cut]) # unlimited reuse of any cut length
return dp[n]

```

Practice Questions — Unbounded Knapsack & Coin Change

Q1. Coin Change (Minimum Coins) Find the fewest coins to make a target amount. *Approach:* $dp[a] = \min$ over coins of $dp[a - \text{coin}] + 1$, filled low to high so a coin can be reused within the same computation.

Q2. Coin Change II (Number of Combinations) Count the number of distinct coin combinations summing to an amount. *Approach:* Loop coins in the OUTER loop and amount in the inner loop so $\{1,2\}$ and $\{2,1\}$ aren't double-counted as different combinations.

Q3. Combination Sum IV (Count Permutations) Count orderings of numbers that sum to a target (order matters). *Approach:* Swap loop order versus Coin Change II — amount outer, numbers inner — so each distinct ORDER is counted separately.

Q4. Rod Cutting Maximize revenue cutting a rod into pieces given a price table. *Approach:* $dp[\text{length}] = \max$ over all first-cut choices of $\text{price}[\text{cut}] + dp[\text{length} - \text{cut}]$ — unbounded because any cut length can be reused any number of times.

Q5. Perfect Squares Find the minimum number of perfect square numbers summing to n . *Approach:* Same shape as Coin Change (min coins), with "coins" being all perfect squares $\leq n$.

Q6. Minimum Cost For Tickets Minimize cost of travel passes (1-day/7-day/30-day) covering a set of travel days. *Approach:* $dp[\text{day}] = \min$ cost to cover travel through `day`, trying each pass type as an unbounded "reach forward" option from the current day.

2.5 Longest Common Subsequence (LCS) Family

What it's for: DP over a PAIR of sequences, where the state is a pair of indices (i, j) — one into each sequence — and the recurrence branches on whether the current characters/elements match.

Use Cases:

- Longest Common Subsequence — diffing tools, version control merge logic.
- Edit Distance — spell checkers, DNA sequence alignment, fuzzy string matching.
- Longest Common Substring (contiguous variant), Shortest Common Supersequence.
- Any problem comparing/aligning two sequences element by element.

Limitations:

- $O(m \times n)$ time and space for two sequences of length m and n — can be too slow/memory-heavy for very long sequences (e.g., whole-genome alignment needs specialized algorithms).
- Space is reducible to $O(\min(m, n))$ with a rolling 2-row array, but then reconstructing the actual subsequence/edit-path requires either keeping the full table or a more clever backtrack-while-computing technique.
- Easy to confuse "subsequence" (elements in order, not necessarily contiguous) with "substring" (must be contiguous) — they use different reset rules on mismatch.

Time Complexity: $O(m \times n)$ — one recurrence evaluation per pair of indices **Space Complexity:** $O(m \times n)$, reducible to $O(\min(m, n))$ with row rolling

Approach (short): Define $dp[i][j]$ as the answer considering the first i characters of string A and the first j characters of string B. On a match, extend the diagonal ($dp[i-1][j-1] + 1$ or $+ 0$ cost); on a mismatch, take the best of skipping a character from either string (or substituting, for edit distance).

Keywords & Constraints: "longest common subsequence", "edit distance", "two strings", "minimum operations to convert", $m, n \leq 10^3$ - 10^4 .

```
def lcs_length(a, b):
    """Longest Common Subsequence length between strings a and b."""
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = 1 + dp[i - 1][j - 1]          # characters match: extend the diagonal
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]) # mismatch: skip a char from either string
    return dp[m][n]

def edit_distance(a, b):
    """Minimum insert/delete/replace operations to turn a into b."""
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i                                     # delete all i characters of a
    for j in range(n + 1):
        dp[0][j] = j                                     # insert all j characters of b
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]             # chars match: no operation needed here
            else:
                dp[i][j] = 1 + min(dp[i - 1][j],        # delete from a
                                   dp[i][j - 1],        # insert into a
                                   dp[i - 1][j - 1])     # replace
    return dp[m][n]

def longest_common_substring(a, b):
```

```

"""Longest CONTIGUOUS common substring -- resets to 0 on mismatch (unlike LCS)."""
m, n = len(a), len(b)
dp = [[0] * (n + 1) for _ in range(m + 1)]
best = 0
for i in range(1, m + 1):
    for j in range(1, n + 1):
        if a[i - 1] == b[j - 1]:
            dp[i][j] = 1 + dp[i - 1][j - 1]
            best = max(best, dp[i][j])
        else:
            dp[i][j] = 0
return best
# RESET on mismatch -- must stay contiguous

```

Practice Questions — Longest Common Subsequence (LCS) Family

Q1. Longest Common Subsequence Find the length of the LCS between two strings. *Approach:* $dp[i][j] = dp[i-1][j-1] + 1$ on a match, else $\max(dp[i-1][j], dp[i][j-1])$ on a mismatch.

Q2. Edit Distance (Levenshtein Distance) Find minimum insert/delete/replace ops to convert one string to another. *Approach:* On a match carry $dp[i-1][j-1]$ forward unchanged; on a mismatch take $1 +$ the best of insert/delete/replace neighbors.

Q3. Longest Common Substring Find the length of the longest CONTIGUOUS common substring. *Approach:* Same table shape as LCS but reset $dp[i][j]$ to 0 on any mismatch, since a substring can't skip characters.

Q4. Shortest Common Supersequence Find the shortest string containing both strings as subsequences. *Approach:* Length = $m + n - \text{LCS}(a, b)$; reconstructing the actual string walks the LCS table taking non-matching characters from whichever string as needed.

Q5. Distinct Subsequences Count how many times string t appears as a subsequence of string s . *Approach:* $dp[i][j] =$ ways using first i chars of s to form first j chars of t : $dp[i-1][j]$ (skip $s[i]$) plus $dp[i-1][j-1]$ (use $s[i]$) when $s[i] == t[j]$.

Q6. Interleaving String Determine if a string is formed by interleaving two other strings while preserving order. *Approach:* $dp[i][j] =$ can first $i+j$ characters of the interleaving be formed from first i of s_1 and first j of s_2 — branches on which source string contributes the current character, a 2-index DP just like LCS.

2.6 Longest Increasing Subsequence (LIS) Family

What it's for: DP where $dp[i]$ represents the best answer for a subsequence ENDING exactly at index i , and the transition looks back at all valid earlier indices $j < i$ that could precede it (usually those with a smaller value).

Use Cases:

- Longest Increasing Subsequence and its many variants (non-decreasing, longest decreasing, etc.).
- Russian Doll Envelopes — LIS extended to 2 dimensions after sorting.
- Maximum sum increasing subsequence, longest chain of pairs.
- Box stacking, building bridges/matchings that require an ordering constraint.

Limitations:

- The direct $O(n^2)$ DP (checking every earlier index j for each i) is too slow for large n ($\geq 10^5$) — the $O(n \log n)$ patience-sorting/binary-search variant is needed at that scale, but changes the technique (no longer a simple "look at all j " DP).
- The $O(n \log n)$ variant's auxiliary "tails" array does NOT directly hold a valid subsequence — only its LENGTH is guaranteed correct; reconstructing the actual subsequence needs extra parent-pointer bookkeeping.
- Many LIS-family problems require a NON-OBVIOUS preprocessing step (like sorting by one dimension, then running LIS on another) to reduce to the base pattern — recognizing that reduction is the hard part.

Time Complexity: $O(n^2)$ for the direct DP; $O(n \log n)$ with binary search on a "smallest tail" array **Space Complexity:** $O(n)$ for the dp array (and the tails array in the optimized version)

Approach (short): Define $dp[i]$ = length (or best value) of the best increasing subsequence ENDING at index i . For each i , look at every $j < i$ with $nums[j] < nums[i]$ and take $dp[i] = \max(dp[i], dp[j] + 1)$. Answer is the max over all $dp[i]$.

Keywords & Constraints: "longest increasing subsequence", "ending at index i ", "chain of pairs", $n \leq 10^3$ - 10^4 for $O(n^2)$; $n \leq 10^5$ - 10^6 needs the $O(n \log n)$ version.

```
def lis_length_on2(nums):
    """ $O(n^2)$  DP:  $dp[i]$  = length of LIS ending exactly at index  $i$ ."""
    n = len(nums)
    if n == 0:
        return 0
    dp = [1] * n
    for i in range(n):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)

# every single element is an LIS of length 1 by itself
# extend the best increasing subsequence ending at j

import bisect

def lis_length_nlogn(nums):
    """ $O(n \log n)$ : maintain `tails[k]` = smallest possible tail value of an increasing subsequence of length  $k+1$ ."""
    tails = []
    for x in nums:
        pos = bisect.bisect_left(tails, x)
        if pos == len(tails):
            tails.append(x)
        else:
            tails[pos] = x
    return len(tails)

# find where x could extend/replace in tails
# x extends the longest subsequence found so far
# x gives a smaller/equal tail for that length -- improves future
# NOTE: tails itself is not a valid LIS, only its length is

def max_sum_increasing_subsequence(nums):
    """ $dp[i]$  = max SUM of an increasing subsequence ending at index  $i$  (variant: optimize sum, not length)."""
    n = len(nums)
    dp = nums[:]
    for i in range(n):
        for j in range(i):
```

```

        if nums[j] < nums[i]:
            dp[i] = max(dp[i], dp[j] + nums[i])
    return max(dp)

```

Practice Questions — Longest Increasing Subsequence (LIS) Family

Q1. Longest Increasing Subsequence Find the length of the LIS in an array. *Approach:* $dp[i] = 1 + \max(dp[j])$ for $j < i$ where $nums[j] < nums[i]$, or $O(n \log n)$ via a binary-search "tails" array for large n .

Q2. Longest Non-Decreasing Subsequence Same as LIS but equal values are allowed to extend the chain. *Approach:* Identical recurrence with a $\leq$ comparison instead of strictly $<$, changing `bisect_left` to `bisect_right` in the $O(n \log n)$ version.

Q3. Maximum Sum Increasing Subsequence Find the increasing subsequence with the maximum sum (not just length). *Approach:* Same $dp[i]$ transition as LIS, but track running SUM instead of running LENGTH, initialized to $nums[i]$ instead of 1.

Q4. Russian Doll Envelopes Find the max number of envelopes nestable inside each other (both dimensions must increase). *Approach:* Sort by width ascending (and height descending for equal widths), then run LIS on the height sequence — the sort neutralizes one dimension, reducing 2D nesting to 1D LIS.

Q5. Longest Chain of Pairs Find the longest chain of pairs (a,b) where each next pair's first element exceeds the previous pair's second. *Approach:* Sort pairs by first element, then it becomes a direct LIS-style DP comparing chainability instead of raw value order.

Q6. Number of Longest Increasing Subsequences Count how many distinct LIS of maximum length exist. *Approach:* Track both $dp[i]$ (best length ending at i) and $count[i]$ (number of ways to achieve that length), updating $count[i]$ by summing counts from all j that tie for the best extension.

2.7 Palindrome & String DP

What it's for: DP over substrings of a single string, where the state is typically a (start, end) index range, used to answer questions about palindromic structure or other substring-range properties.

Use Cases:

- Checking/counting palindromic substrings, longest palindromic substring.
- Minimum insertions/deletions to make a string a palindrome.
- Minimum cuts to partition a string into palindromic pieces.
- General "is substring [i, j] valid under some property" precomputation used by other algorithms (e.g., backtracking partition).

Limitations:

- $O(n^2)$ states (all substrings) is standard for this family — becomes slow for very long strings ($n > \sim 5000$ for the full $O(n^2)$ table).
- Must fill the table in order of increasing substring LENGTH (not simply row by row) since $dp[i][j]$ depends on $dp[i+1][j-1]$, a shorter, "more inward" substring.
- Some problems (like minimum palindrome cuts) need a SECOND 1D DP layered on top of the palindrome table — recognizing when one DP feeds another is key.

Time Complexity: $O(n^2)$ to build the palindrome table; some derived problems add an extra $O(n)$ or $O(n^2)$ layer on top **Space Complexity:** $O(n^2)$ for the substring-range table

Approach (short): Define $dp[i][j]$ = True/cost for the substring $s[i..j]$. Base cases: single characters are palindromes (length 1), and pairs are palindromes if the two characters match (length 2). For longer ranges, $s[i..j]$ is a palindrome iff $s[i]==s[j]$ AND $dp[i+1][j-1]$ is a palindrome — fill by increasing length.

Keywords & Constraints: "palindromic substring", "minimum insertions/deletions to make a palindrome", "partition into palindromes", $n \leq 10^3$ -2000.

```
def palindrome_table(s):
    """dp[i][j] = True iff s[i..j] (inclusive) is a palindrome. Filled by increasing substring length."""
    n = len(s)
    dp = [[False] * n for _ in range(n)]
    for i in range(n):
        dp[i][i] = True
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            if s[i] != s[j]:
                dp[i][j] = False
            elif length == 2:
                dp[i][j] = True
            else:
                dp[i][j] = dp[i + 1][j - 1]
    return dp

def longest_palindromic_substring(s):
    dp = palindrome_table(s)
    best = ""
    n = len(s)
    for i in range(n):
        for j in range(i, n):
            if dp[i][j] and (j - i + 1) > len(best):
                best = s[i:j + 1]
    return best

def min_insertions_to_make_palindrome(s):
    """Minimum insertions needed -- equals n - LCS(s, reverse(s))."""
    rev = s[::-1]
```

```

n = len(s)
dp = [[0] * (n + 1) for _ in range(n + 1)]
for i in range(1, n + 1):
    for j in range(1, n + 1):
        if s[i - 1] == rev[j - 1]:
            dp[i][j] = 1 + dp[i - 1][j - 1]
        else:
            dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
lcs = dp[n][n]
return n - lcs

def min_palindrome_cuts(s):
    """Minimum cuts to partition s so every piece is a palindrome."""
    is_pal = palindrome_table(s)
    n = len(s)
    cuts = [0] * n
    for i in range(n):
        if is_pal[0][i]:
            cuts[i] = 0
            # whole prefix s[0..i] is already a palindrome
        else:
            cuts[i] = min(cuts[j] + 1 for j in range(i) if is_pal[j + 1][i])
    return cuts[-1]

```

Practice Questions — Palindrome & String DP

Q1. Longest Palindromic Substring Find the longest substring that is itself a palindrome. *Approach:* Build the full (i,j) palindrome table in increasing length order, then scan for the longest True range.

Q2. Palindromic Substrings (Count) Count the total number of palindromic substrings. *Approach:* Same palindrome table construction, but increment a counter for every True cell instead of tracking the longest.

Q3. Minimum Insertions to Make a String Palindrome Find fewest character insertions needed to make s a palindrome. *Approach:* Equals n minus the LCS of s and its reverse — combines the LCS pattern with a palindrome insight.

Q4. Minimum Deletions to Make a String Palindrome Find fewest deletions needed to make s a palindrome. *Approach:* Identical formula/approach to minimum insertions (n - LCS(s, reverse(s))) since inserting and deleting are symmetric for this problem.

Q5. Palindrome Partitioning II (Minimum Cuts) Find the minimum number of cuts to partition s into all-palindrome pieces. *Approach:* Precompute the palindrome table, then run a 1D DP cuts[i] = min cuts for the FIRST i+1 characters, checking every valid palindromic suffix ending at i.

Q6. Longest Palindromic Subsequence Find the length of the longest subsequence (not necessarily contiguous) that is a palindrome. *Approach:* Equals LCS(s, reverse(s)) — reduces a palindrome-subsequence question directly to the LCS family via the same s/reverse(s) trick.

2.8 Interval DP

What it's for: DP over sub-ranges $[i, j]$ of a sequence where the optimal answer for a range depends on trying every possible "split point" k inside that range and combining the best answers for the two resulting sub-ranges.

Use Cases:

- Matrix Chain Multiplication — minimize scalar multiplications when multiplying a chain of matrices.
- Burst Balloons — maximize coins from bursting balloons in some order.
- Minimum cost to merge stones/piles into one pile.
- Optimal Binary Search Tree construction, polygon triangulation cost minimization.

Limitations:

- $O(n^3)$ time is standard for this family ($O(n^2)$ ranges, each trying $O(n)$ split points) — becomes slow beyond roughly $n \leq 300-500$.
- MUST fill in order of increasing range LENGTH, since $dp[i][j]$ depends on $dp[i][k]$ and $dp[k+1][j]$ for various k strictly inside — filling row by row instead of length by length gives wrong answers.
- Figuring out the correct "combine at split k " cost formula is the hard part of each problem and differs significantly between problems (unlike, say, Knapsack's very uniform recurrence).

Time Complexity: $O(n^3)$ — $O(n^2)$ ranges $\times$ $O(n)$ split points tried per range **Space Complexity:** $O(n^2)$ for the range table

Approach (short): Define $dp[i][j]$ = optimal cost/value for the range $[i, j]$. For every split point k with $i \leq k < j$, compute $dp[i][k] + dp[k+1][j] + \text{cost_of_combining_at_}k$, and take the best over all k . Fill by increasing $(j - i)$ range length.

Keywords & Constraints: "minimum cost to multiply/merge/combine a sequence", "order of operations matters", "matrix chain", "burst balloons", $n \leq 300-500$.

```
def matrix_chain_order(dims):
    """dims: [p0, p1, ..., pn] where matrix i has dimensions p[i-1] x p[i].
    Returns minimum scalar multiplications to multiply the full chain."""
    n = len(dims) - 1                # number of matrices
    dp = [[0] * n for _ in range(n)]
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            dp[i][j] = float('inf')
            for k in range(i, j):
                # try every split point between i and j
                cost = dp[i][k] + dp[k + 1][j] + dims[i] * dims[k + 1] * dims[j + 1]
                dp[i][j] = min(dp[i][j], cost)
    return dp[0][n - 1]

def burst_balloons_max_coins(nums):
    """Maximize coins bursting balloons; bursting i gives nums[left]*nums[i]*nums[right]."""
    balloons = [1] + nums + [1]      # pad with virtual 1-valued balloons at both ends
    n = len(balloons)
    dp = [[0] * n for _ in range(n)]
    for length in range(2, n):
        # length = distance between the padding boundaries
        for i in range(0, n - length):
            j = i + length
            for k in range(i + 1, j):
                # k = last balloon burst WITHIN the OPEN range (i, j)
                coins = balloons[i] * balloons[k] * balloons[j] + dp[i][k] + dp[k][j]
                dp[i][j] = max(dp[i][j], coins)
    return dp[0][n - 1]

def min_cost_merge_stones(stones, k):
    """Simplified sketch: minimum cost to merge adjacent piles (pairwise, k=2) into one, cost = sum merged."""
    n = len(stones)
    prefix = [0] * (n + 1)
    for i, v in enumerate(stones):
        prefix[i + 1] = prefix[i] + v
```

```

dp = [[0] * n for _ in range(n)]
for length in range(2, n + 1):
    for i in range(n - length + 1):
        j = i + length - 1
        dp[i][j] = min(dp[i][m] + dp[m + 1][j] for m in range(i, j)) + (prefix[j + 1] - prefix[i])
return dp[0][n - 1]

```

Practice Questions — Interval DP

Q1. Matrix Chain Multiplication Find the minimum number of scalar multiplications to multiply a chain of matrices.

Approach: Try every split point k between i and j as the "last multiplication" performed, combining the cost of the two resulting sub-chains plus the cost of multiplying their results together.

Q2. Burst Balloons Maximize coins earned bursting balloons in some order. *Approach:* Think in reverse: $dp[i][j]$ considers which balloon is burst LAST within the open range (i, j) , since that balloon's neighbors at burst time are exactly i and j .

Q3. Minimum Cost to Merge Stones Find minimum cost to merge piles of stones (k adjacent piles at a time) into one pile. *Approach:* Interval DP over which sub-ranges get fully merged first, with an added divisibility condition on range length based on k .

Q4. Optimal Binary Search Tree Arrange keys with known access frequencies into a BST minimizing expected search cost. *Approach:* $dp[i][j] = \min \text{ over root choices } k \text{ in } [i, j] \text{ of } dp[i][k-1] + dp[k+1][j] + \text{sum of frequencies in } [i, j]$ — the "combine" cost accounts for every node's depth increasing by one under a new root.

Q5. Palindrome Partitioning (Minimum Cuts, Interval View) Revisit minimum palindrome cuts as a pure interval DP. *Approach:* $dp[i][j] = \text{cost to fully palindrome-partition } s[i..j]$, trying every split point k and combining with a precomputed palindrome-range check.

Q6. Polygon Triangulation (Minimum Score) Triangulate a convex polygon minimizing the sum of triangle vertex-value products. *Approach:* Same "try every split point k " interval DP as Matrix Chain, with the combine cost being the product of the three vertices of the triangle formed by edge (i, j) and vertex k .

2.9 DP on Trees

What it's for: Combines tree recursion with DP: each node computes a DP value from its children's DP values (post-order), often needing MULTIPLE related values per node (e.g., "best if this node is included" and "best if excluded") to correctly combine results.

Use Cases:

- House Robber III — maximize value on a tree where adjacent (parent-child) nodes can't both be picked.
- Diameter of a tree, maximum path sum through any node.
- Counting subtrees with a given property, tree coloring/matching problems.
- Rerooting techniques — computing an answer for EVERY node as if it were the root, efficiently.

Limitations:

- Requires post-order traversal (children before parent) since a node's DP value depends on its children's — accidentally processing top-down instead of bottom-up breaks the recurrence.
- Problems needing "best answer if this node is included" AND "best if excluded" require returning a PAIR (or small tuple) from each recursive call, not a single number — easy to under-model the state.
- Recursion depth equals tree height; a skewed tree can approach $O(n)$ depth and risk Python's recursion limit for very large trees.

Time Complexity: $O(n)$ — each node processed once, $O(1)$ work combining a constant number of children's values (for binary trees) or $O(\text{children})$ for general trees **Space Complexity:** $O(h)$ recursion stack (h = tree height), plus $O(n)$ if storing a result per node

Approach (short): Post-order traverse the tree; at each node, first recursively get the DP value(s) from left and right (or all) children, then combine them with the current node's value to compute this node's DP value(s), which get returned up to the parent.

Keywords & Constraints: "binary tree", "cannot select both parent and child", "maximum path sum in a tree", "diameter", $n \leq 10^4$ - 10^5 nodes.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def house_robber_iii(root):
    """Max value picking nodes with no two adjacent (parent-child) both selected."""
    def dfs(node):
        if node is None:
            return (0, 0) # (best if excluded, best if included) -- both 0 for empty
        left_excl, left_incl = dfs(node.left)
        right_excl, right_incl = dfs(node.right)
        excl = max(left_excl, left_incl) + max(right_excl, right_incl) # skip node: children free to choose either
        incl = node.val + left_excl + right_excl # take node: children MUST be excluded
        return (excl, incl)
    return max(dfs(root))

def max_path_sum_any_node(root):
    """Maximum path sum where the path can start/end anywhere (not necessarily root-to-leaf)."""
    best = [float('-inf')]
    def dfs(node):
        if node is None:
            return 0
        left_gain = max(dfs(node.left), 0) # ignore negative contributions from a child
        right_gain = max(dfs(node.right), 0)
        best[0] = max(best[0], node.val + left_gain + right_gain) # path THROUGH this node (both children)
        return node.val + max(left_gain, right_gain) # path CONTINUING UP can only use one side
    dfs(root)
    return best[0]
```

```
def diameter_of_tree(root):
    """Longest path between any two nodes, in EDGES."""
    best = [0]
    def depth(node):
        if node is None:
            return 0
        l, r = depth(node.left), depth(node.right)
        best[0] = max(best[0], l + r)
        return 1 + max(l, r)
    depth(root)
    return best[0]
```

Practice Questions — DP on Trees

Q1. House Robber III Maximize value on a tree where a parent and its child can't both be selected. *Approach:* Each node returns a pair (best-excluding-self, best-including-self); combining pairs from left/right children lets the parent correctly enforce the no-adjacent-selection rule.

Q2. Binary Tree Maximum Path Sum Find the max sum path between any two nodes in a binary tree. *Approach:* Each node computes the best "one-sided" extension to pass up to its parent, while separately tracking the best "through this node using both sides" as a global answer.

Q3. Diameter of Binary Tree Find the longest path (in edges) between any two nodes. *Approach:* While computing subtree depth bottom-up, track a running max of left-depth + right-depth at every node — the path through that node.

Q4. Count Good Nodes in Binary Tree Count nodes where no ancestor has a greater value. *Approach:* Pass the maximum value seen so far DOWN the recursion (not up) — a tree DP that threads state from parent to child rather than child to parent.

Q5. Sum of Distances in Tree (Rerooting) For every node, find the sum of distances to all other nodes. *Approach:* First a bottom-up pass computes subtree sizes and distance sums for one root; a second top-down "rerooting" pass adjusts each child's answer from its parent's in $O(1)$, avoiding an $O(n)$ recomputation per node.

Q6. Binary Tree Cameras Find the minimum number of cameras needed to monitor every node. *Approach:* Each node returns one of three states (uncovered / covered-no-camera / has-camera) computed from its children, with a greedy-DP hybrid rule choosing to place a camera only when a child would otherwise be left uncovered.

2.10 Bitmask DP

What it's for: DP where part of the state is a SET (which items/cities/tasks have been used/visited so far), represented as an integer bitmask so set membership, union, and iteration are all $O(1)/O(n)$ bitwise operations instead of $O(n)$ array/set operations.

Use Cases:

- Traveling Salesman Problem (TSP) — exact DP solution in $O(2^n \times n^2)$ instead of brute-force $O(n!)$.
- Assignment problems — assign n tasks to n workers minimizing total cost.
- Counting/optimizing over subsets where "which elements have been used" is essential state.
- Any problem where a backtracking bitmask (see Backtracking handbook, 2.8) would repeat the same subset across different call paths — the signal to switch to bitmask DP.

Limitations:

- Practically limited to $n \leq \sim 20-22$, since the state space is $O(2^n)$ — far too large beyond that for either time or memory.
- dp arrays are typically 2D: $dp[mask][last]$ — $O(2^n \times n)$ states, each transition $O(n)$, giving $O(2^n \times n^2)$ total, which is the practical ceiling on n .
- Bit manipulation itself (extracting bits, iterating set bits, checking membership) is a skill of its own and a common source of off-by-one/logic bugs.

Time Complexity: $O(2^n \times n^2)$ typical (TSP-style); $O(2^n \times n)$ for simpler subset-only state **Space Complexity:** $O(2^n \times n)$ for the dp table

Approach (short): Define $dp[mask][last]$ = best answer having used exactly the set of items in `mask`, currently at/last-placed item `last`. Transition by trying every item NOT in mask as the next one to add, moving to $dp[mask | (1 \ll next)][next]$.

Keywords & Constraints: "visit all cities", "assign each task to exactly one worker", "subset state", $n \leq 15-20$ explicitly small in constraints (a strong signal for bitmask DP).

```
def tsp_bitmask_dp(dist):
    """Minimum cost to visit all cities starting and ending at city 0. dist[i][j] = cost from i to j."""
    n = len(dist)
    FULL = (1 << n) - 1
    dp = [[float('inf')] * n for _ in range(1 << n)]
    dp[1][0] = 0 # mask=1 (only city 0 visited), currently at city 0

    for mask in range(1 << n):
        for last in range(n):
            if dp[mask][last] == float('inf') or not (mask & (1 << last)):
                continue # unreachable state, skip
            for nxt in range(n):
                if mask & (1 << nxt):
                    continue # already visited nxt
                new_mask = mask | (1 << nxt)
                new_cost = dp[mask][last] + dist[last][nxt]
                if new_cost < dp[new_mask][nxt]:
                    dp[new_mask][nxt] = new_cost

    return min(dp[FULL][last] + dist[last][0] for last in range(n)) # return to city 0 at the end

def min_cost_assignment(cost):
    """Assign n tasks to n workers (cost[worker][task]) minimizing total cost."""
    n = len(cost)
    dp = [[float('inf')] * (1 << n) for _ in range(1 << n)]
    dp[0] = 0 # no tasks assigned yet
    for mask in range(1 << n):
        if dp[mask] == float('inf'):
            continue
        # ... (rest of the function code) ...
```

```

        worker = bin(mask).count("1")                # number of tasks assigned so far = which worker goes next
        if worker == n:
            continue
        for task in range(n):
            if not (mask & (1 << task)):
                new_mask = mask | (1 << task)
                dp[new_mask] = min(dp[new_mask], dp[mask] + cost[worker][task])
        return dp[(1 << n) - 1]

def can_partition_k_subsets_bitmask(nums, k):
    """Bitmask-DP version of Partition into K Equal Sum Subsets: dp[mask] = remaining capacity in current bucket."""
    total = sum(nums)
    if total % k != 0:
        return False
    target = total // k
    n = len(nums)
    dp = [-1] * (1 << n)                # dp[mask] = remaining space in the bucket currently being filled
    dp[0] = target
    for mask in range(1 << n):
        if dp[mask] < 0:
            continue
        for i in range(n):
            if mask & (1 << i) or nums[i] > dp[mask]:
                continue
            new_mask = mask | (1 << i)
            new_remaining = dp[mask] - nums[i]
            dp[new_mask] = target if new_remaining == 0 else new_remaining
    return dp[(1 << n) - 1] >= 0

```

Practice Questions — Bitmask DP

Q1. Traveling Salesman Problem (Exact DP) Find the minimum cost Hamiltonian cycle visiting all cities. *Approach:* $dp[mask][last] = \min$ cost having visited exactly `mask`, ending at `last`; transition tries every unvisited next city, giving $O(2^n n^2)$ instead of brute-force $O(n!)$.

Q2. Assignment Problem Assign n tasks to n workers one-to-one minimizing total cost. *Approach:* $dp[mask] = \min$ cost assigning tasks in `mask`, where $\text{popcount}(mask)$ implicitly tells you which worker is being assigned next.

Q3. Partition to K Equal Sum Subsets (Bitmask DP) Determine if an array can be split into k equal-sum subsets. *Approach:* $dp[mask] = \text{remaining capacity in the bucket currently being filled after using exactly the elements in `mask`}$ — revisits the backtracking version of this problem with memoized bitmask state.

Q4. Shortest Path Visiting All Nodes Find the shortest path in a graph that visits every node at least once. *Approach:* BFS/DP over (current node, visited mask) states, since the graph allows revisiting nodes — bitmask tracks progress toward "all nodes visited" as the goal condition.

Q5. Fluid/Task Scheduling to Minimize Maximum Load Distribute tasks among workers minimizing the maximum load on any worker. *Approach:* $dp[mask] = \text{best (min possible max-load) way to assign tasks in `mask`}$, combined with binary search on the answer or direct min-max tracking per mask.

Q6. Count Number of Ways to Wear Different Hats to Each Other Assign each of n people a distinct hat from their preference list. *Approach:* $dp[mask]$ over "which people have been assigned a hat so far", iterating hats in the outer loop and, for each hat, deciding which (if any) eligible unassigned person receives it — a bitmask DP over people rather than over hats directly.

PART 3: QUICK REFERENCE CHEAT SHEET

Pattern	Purpose	Time	Space	Fails When
1D / Linear DP	Answer at i from $i-1$, $i-2$...	$O(n)$	$O(n) \rightarrow O(1)$	State needs more than a few prior indices
2D Grid DP	Path count/cost on a grid	$O(r \times c)$	$O(r \times c) \rightarrow O(c)$	Movement isn't simply from above/left
0/1 Knapsack	Pick each item at most once	$O(n \times \text{cap})$	$O(\text{cap})$	Capacity too large for a table
Unbounded Knapsack	Pick items unlimited times	$O(n \times \text{amount})$	$O(\text{amount})$	Confusing loop order with 0/1 (high-to-low vs low-to-high)
LCS Family	Compare/align two sequences	$O(m \times n)$	$O(m \times n) \rightarrow O(\min(m, n))$	Sequences too long for $O(m \times n)$
LIS Family	Best chain ending at i	$O(n^2)$ or $O(n \log n)$	$O(n)$	n too large for $O(n^2)$; needs the $\log n$ variant
Palindrome / String DP	Property of substring $[i, j]$	$O(n^2)$	$O(n^2)$	n too large for an $O(n^2)$ table
Interval DP	Best way to combine a range via a split	$O(n^3)$	$O(n^2)$	n too large for $O(n^3)$ (roughly $n > 300-500$)
DP on Trees	Combine children into parent	$O(n)$	$O(h)$	Needs global answer requiring 2 passes (rerooting)
Bitmask DP	Track a used/visited subset	$O(2^n \times n^2)$	$O(2^n \times n)$	n larger than $\sim 20-22$

Decision Guide — “Which Pattern Do I Use?”

1. Answer at position i depends only on a few previous positions in ONE sequence? $\rightarrow$ 1D / Linear DP
2. Problem is on a 2D grid/matrix with moves from adjacent cells? $\rightarrow$ 2D Grid DP
3. Choosing a subset of items under a capacity, each usable at most once? $\rightarrow$ 0/1 Knapsack
4. Same, but items can be reused unlimited times? $\rightarrow$ Unbounded Knapsack & Coin Change
5. Comparing or aligning TWO sequences (strings/arrays)? $\rightarrow$ LCS Family
6. Need the best "chain" of elements that must stay increasing/ordered? $\rightarrow$ LIS Family
7. Property depends on a RANGE/substring $[i, j]$ of ONE sequence (often palindrome-related)? $\rightarrow$ Palindrome & String DP
8. Best way to combine a range requires trying every possible split point k inside it (order of operations matters)? $\rightarrow$ Interval DP
9. Problem is on a TREE and each node's answer depends on its children's answers? $\rightarrow$ DP on Trees

10. "Which elements have been used/visited" IS the state, and n is small ($\leq \sim 20-22$)? $\rightarrow$ Bitmask DP

11. Subproblems do NOT actually overlap (each state reached via only one path)? $\rightarrow$ You likely don't need DP — plain recursion/divide-and-conquer (see Recursion handbook) or backtracking (see Backtracking handbook) is enough